The neural-network bot for Troll Farm

Where the project stands on 2026-08-30

Prepared for the owner by the coordinator (`local_claude_1`)

2026-08-30, 12:5xZ (second edition; the first was 06:1xZ) — everything below is on `main` unless a path says otherwise

In one page

The goal. A bot for the CodinGame game *Troll Farm* whose every command comes from a small neural network — the way the #1 player on the ladder (*delineate*, rating 30.9) plays. Our best hand-written bot ("the champion of record") reads about 18–21 on the same ladder.

The target you set (2026-08-29). A network candidate that beats the champion of record *and* the "orchard 6" bot on our own bench — at least 60 % wins over 400 games against each, three checks in a row — exported as one Rust file under 100,000 characters at $\leq$ 15 ms a turn, verified, ready for the ladder, **by 2026-10-17**. Milestone for your read: the clone playing whole games against the champion's file, aimed at 2026-09-14.

Where we are, two days later.

- The training environment (a full copy of the game that plays 128 games at once) is built, tested and reproduced by a second agent.
- Every recorded game of the top four players has been turned into a training set: **817,811 decisions from 748 games**, 14 MB.
- A first network, **the clone**, learned by copying those decisions. It plays complete, legal games and **beat the champion's own file in 9 of 48 games** (134 points to 186 on average) — *the milestone, reached two weeks early.* Its games are on file for you (Section 7).
- **Self-play training** started at 04:45Z today and was restarted three times before noon: twice for defects the audits found in the trainer, once because the first hours taught the most important lesson of the day — *training against our weak practice bots made the network worse against the champion.* Since 09:42Z the run of record trains against **an exact in-process copy of the champion**, built and reproduced within a day.
- **The cluster is in use.** On your "gpu" the YT launcher ran a smoke job to completion, and **four 12-hour training jobs** now run there in parallel (a sweep over seed, anchor strength and opponent mix), while the run of record continues on this computer.
- Phase 4's engineering — the network as one Rust file, bedded move for move against the clone — is chartered and under way, so a passing candidate ships in hours.

What is next. Every 500 updates the run of record's snapshot is tested against the champion's file; the four cluster jobs' final snapshots are tested the same way when they return (about 01:00Z tomorrow). The clone's 9 wins of 48 is the bar; a snapshot above 55 % starts the card's 400-game checks against the champion and orchard 6. Then the single-file export, the timing and size checks, and — on your word, when the platform is free — one hour on the ladder.

1 The plan and its status

The work is one card, `coordination/tasks/20260829-nn-bot-way-b.md`, in five phases. Each phase has a "done" condition, a "dead" condition and a budget, like every card on the board.

	what	budget	status
0	the software runtime on this computer; the old (July) pipeline re-run end to end	1–2 days	**done** 08-29 14:2xZ
1	the full-game training environment (Rust), with a Python wrapper	6 days	**done** 08-29 21:1xZ, in one day; reproduced
2	the training set, the bench (our judge), the clone	7 days	**tools done and reproduced** 08-30 03:2xZ; **the clone's games on file** 04:5xZ — the milestone
3	self-play training from the clone, with checks every few hours	2–4 weeks	**running**: the run of record since 08-30 09:42Z against the exact champion; four 12-hour cluster jobs since 12:3xZ
4	export to one Rust file, the timing and size checks, one ladder hour	3 days + 1 hour	**engineering chartered** 08-30 11:1xZ (built against the clone); the ladder hour waits for a candidate and for the platform

2 What was done, step by step

2026-08-29 — the analysis and your decisions

I read what the repository already knew: the #1 player's own description of his network (we hold his text), our July attempts at learning (they stopped for reasons that are now avoided), the game engine's speed and accuracy, the recorded games. Conclusion: every piece existed except the assembly. You decided: open the line; way B — *copy the top players first, then improve by self-play*; you judge the clone's games yourself.

2026-08-29 — the environment (Phase 1)

codex_1 built the environment on the VM in a day. Its written interface (what the network sees, how commands are encoded, how a turn is split into decisions) was audited the same day by *chatgpt_1*, on your instruction, and **ten real defects** were found and fixed before any number was accepted — among them: the starting troll was created with the wrong strength on both sides of the test, so the test could not have caught it; a counter of illegal commands that could never be non-zero; a check of *whether* a game was replayed right but not *when* it ended; and a vocabulary of buyable trolls copied from delineate that the other top players do not respect (see Section 4). The final version passed its check — 1,000 self-play games replayed through an independent copy of the rules with no disagreement — and *claude_1* reproduced it byte for byte.

2026-08-29/30 — the training set, the bench, the clone (Phase 2)

claude_1 built the three tools; *codex_1* reproduced them; I ran the training set on this computer (1 minute 42 seconds) and trained the clone (4 passes over the data, 2 hours). The bench — the clone against the champion's actual submitted file, on 24 real ladder maps, once on each side — gave the milestone numbers. Figures 2–4.

2026-08-30 — self-play (Phase 3), and what the first day of it taught

Self-play works like this: the network plays 128 games at once on real ladder maps, is nudged toward what wins, and a leash ("the anchor") keeps it close to the clone at first. Four runs were started today; three are exploratory, one is the run of record.

- **04:45Z, run A** — against six of our hand-written bots and frozen copies of itself. Its win rate against that mix rose from 0 to 42 % in three hours. *But its snapshot lost to the champion's file worse than the clone it started from*: 2 wins of 48 against the clone's 9, 87 points against

134. Meanwhile the audits found two defects in the trainer: the reward of a turn was still being discounted once per troll inside the turn, and the "which troll am I saving for" planes shifted the network's plan decision at the hand-over from copying to self-play. Both fixed.

- **07:40Z, run B** and **08:39Z, run C** — restarted from the clone with the fixes (run C once a third such plane was found). Run C's snapshot, still trained against the old practice mix, again lost to the champion worse than the clone: 3 wins of 48. The lesson was now certain: *what the network trains against matters more than how fast it trains*; our practice bots teach habits that do not transfer.
- **09:42Z, run D — the run of record.** codex_1 was chartered at 07:45Z to build an exact in-process copy of the champion as a training opponent; it delivered at 09:12Z — the copy matches the champion's submitted file on 200 of 200 games and every one of their 49,945 turns — and claude_1 reproduced the proof by 10:02Z. Run D trains against that copy (weighted highest in its pool) from the clone, with all fixes. Its win rate against a pool the champion dominates is a harder number than run A's; the test that matters is its snapshot against the champion's file, every 500 updates. **Its first snapshot (update 500, 2 million decisions) won 3 of 48** — the same early dip as the exploratory runs. So the first hours of self-play cost the network some of the clone's competence before it learns to win, whatever it trains against; whether run D climbs back past the clone's 9 is the question the next snapshots answer, and the four cluster jobs (one with a stronger leash to the clone) are the other half of that experiment.

The cluster (YT). You said one run of 3.5 days is a job for the cluster. July's launcher was adapted to the new trainer (CPU-only, 15 tests, a 2.8 MB payload; PyTorch reused from the 8.5 GB July put in Cypress). The CPU tree's pools refuse immediate operations; on your "gpu" the jobs run in July's GPU tree with one card reserved and unused. The smoke job ran to completion at 899 decisions a second on 16 cores (this computer: about 640 on 14 threads). **Four 12-hour jobs** run there now, 32 cores each, 60 million decisions each, all from the clone against the exact champion: the run-of-record recipe (A), a stronger anchor (B), a champion-heavy pool (C), a self-play-heavy pool (D). Their final snapshots return about 01:00Z and are tested here.

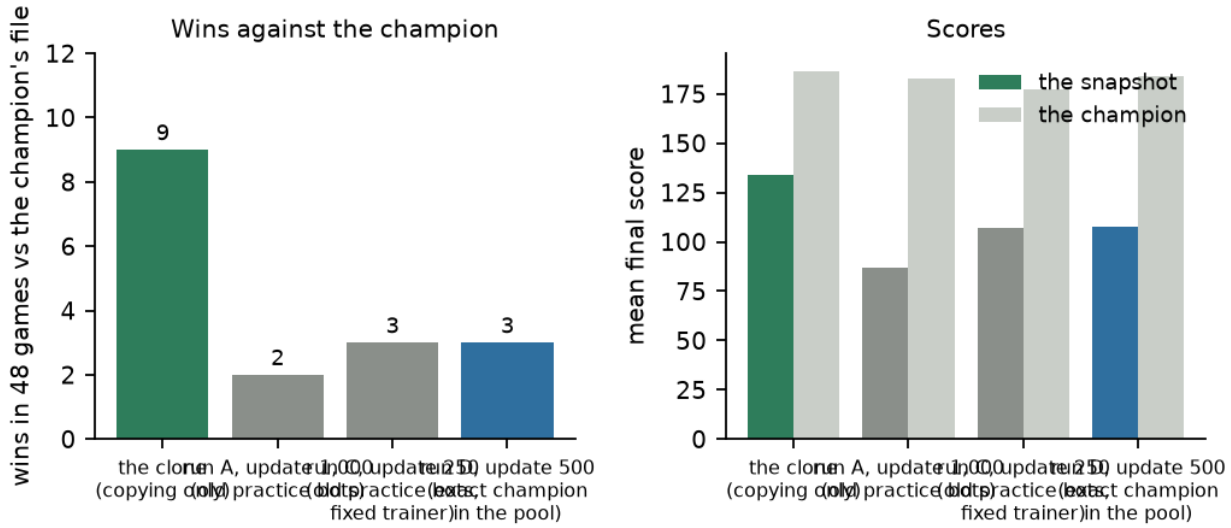


Figure 1: The lesson of the day. Each bar: a network snapshot's wins in 48 games against the champion's own file. The clone (copying only) won 9. Every early self-play snapshot so far won fewer — the runs against our weak practice bots, and the first snapshot of the run of record (D) against the exact champion alike. The early hours of self-play cost competence before they buy any; the question is whether the later snapshots climb past the clone.

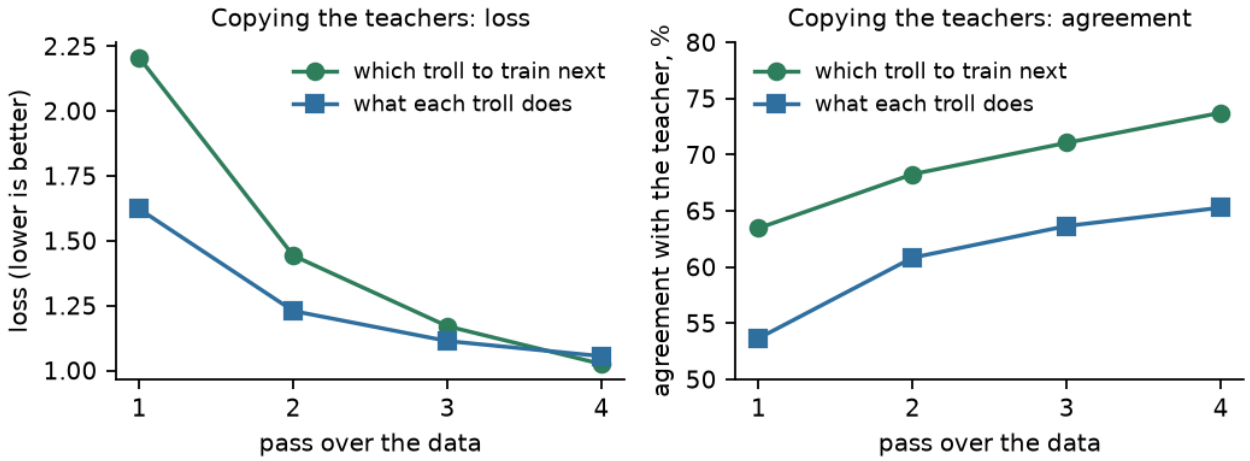


Figure 2: The clone learning to copy the teachers. Left: the loss (an error measure) of its two decisions — which troll to save for, and what each troll does — over the four passes. Right: how often it agrees with the teacher on the training moves.

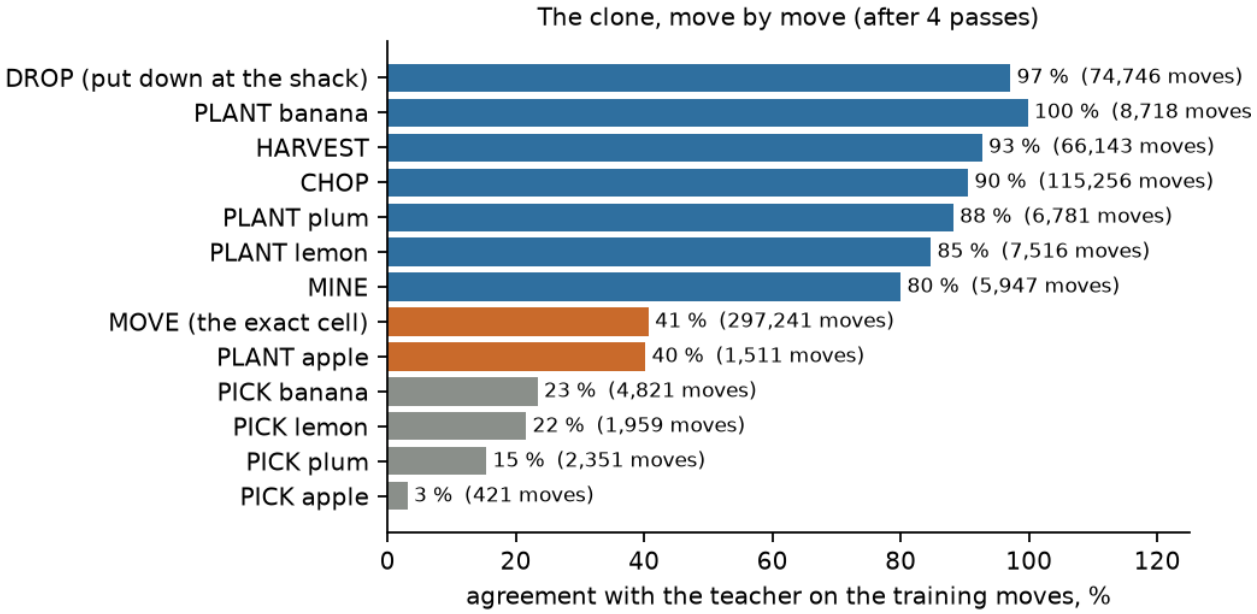


Figure 3: Agreement with the teacher, move by move. The routine moves are learned (drop, harvest, chop, plant); *move to exactly this cell* is the hard one (one cell of up to 242); *pick* (taking a fruit from the shack to plant it) is barely learned — it is rare and depends on a plan the clone does not have.

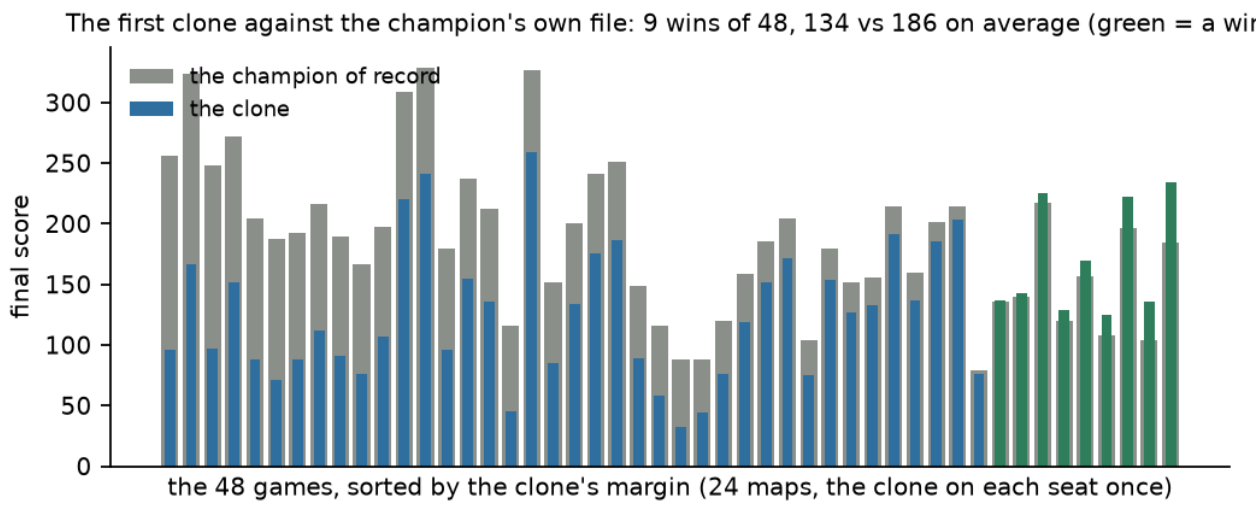


Figure 4: The bench: the clone against the champion's own submitted file, 48 games. Grey bars are the champion's scores, the narrow bars the clone's; green marks the clone's wins. A random-move policy scores 13.5 on this bench.

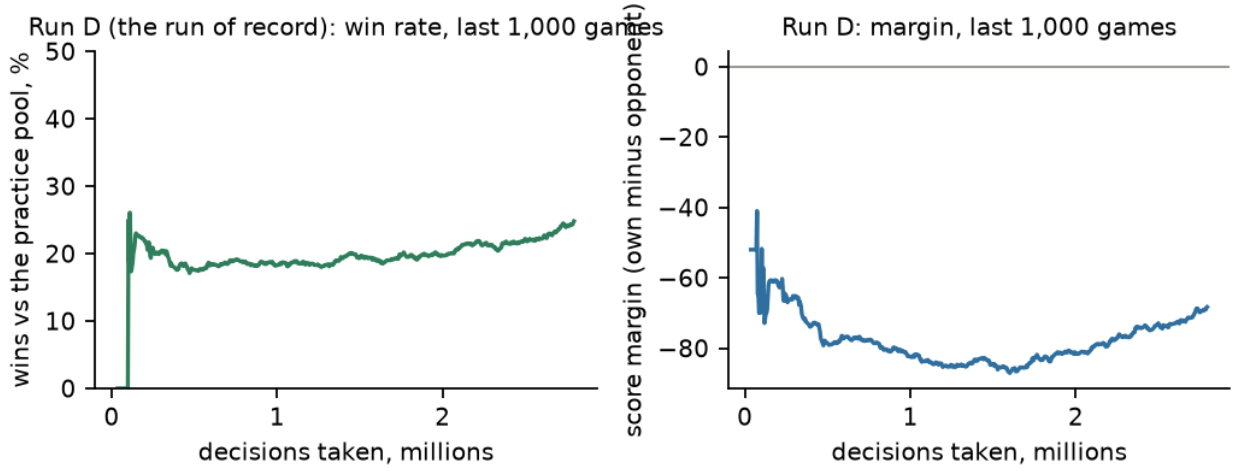


Figure 5: The run of record (run D) so far: trained against a pool the exact champion dominates, so its practice win rate is a harder number than run A's was. Both curves are over the last 1,000 finished games; the test that counts is Figure 1.

3 Where everything is

Two places: the repository (code, cards, reports — on `main`), and a data directory outside it (the big and the reproducible files).

The repository `/home/tarstars/prj/troll_farm` (the same tree as my worktree `.../troll_farm-local_claude_1`)

path	what it is
`coordination/BOARD.md`	the one page to read: what is in motion, your queue
`coordination/GOAL.md`	the target in full, and what I may and may not do without you
`coordination/tasks/20260829-nn-bot-way-b.md`	the card: the design, every ruling, the log — the record of this line
`coordination/tasks/20260829-nn-bot-way-b-env.md`, `...-dataset.md`	the two builders' contracts (Phases 1 and 2)
`local_claude_1/nn-bot/ANALYSIS-2026-08-29.md`	the analysis you read before deciding
`local_claude_1/nn-bot/OBS-PLANES.md`, `ENV-API.md`	the signed interface: the 104 things the network sees; the environment's calls
`rust/src/rl_full.rs`	the training environment (Rust): the game, 128 copies at once, the opponents
`cgauto/rl_full_env.py`	its Python side: `rewards, info = env.step(actions)`; the replay checker
`cgauto/train_level1_ppo.py`	the network itself (`SpatialActorCritic`) with its two heads
`local_claude_1/nn-bot/nn_runtime.py`	one shared adapter: the board to planes, the masks, the command codec, the end rule
`local_claude_1/nn-bot/build_dataset.py`	recorded games → the training set
`local_claude_1/nn-bot/train_clone.py`	the training set → the clone
`local_claude_1/nn-bot/train_ppo_full.py`	the clone → a better network by self-play
`local_claude_1/nn-bot/bench.py`	the judge: a network against a compiled bot on real maps; prints any game turn by turn
`local_claude_1/nn-bot/fake_full_env.py`	a stand-in environment for tests
`local_claude_1/nn-bot/results/clone-2026-08-30-a/`	**the clone, its 48 games, the reports, a README for your read**
`rust/src/strategies/champion_exact.rs`	the exact in-process copy of the champion (a training opponent), generated from the champion's own source; its proof in `codex_1/results/nn-bot-way-b-champion/`
`local_claude_1/nn-bot/yt_ppo_launcher.py`, `yt_ppo_entrypoint.py`	the cluster launcher (prepare / start / monitor / retrieve) and what runs inside a job
`coordination/tasks/20260829-nn-bot-way-b-champion.md`, `...-export.md`	the champion-opponent contract (done) and Phase 4's engineering contract (in progress)
`local_claude_1/reconstructions/fits/reconstruct.py`	the exact rebuild of a recorded game, turn by turn
`tests/test_rl_full_env.py`, `tests/test_train_ppo_full.py`	the tests (7 and 42)
`docs/reports/2026-08-30-neural-network-line-progress.pdf`	this document

The data directory `/home/tarstars/nn-data/` (outside the repository)

path	what it is
`dataset-v400-2026-08-30/`	the training set: 817,811 decisions, 14 MB, with checksums
`clone-2026-08-30-a/`	the clone (`clone-pilot.pt`), its training log, both bench reports and the 48 games (also copied into the repository)
`ppo-2026-08-30-a/`, `-b/`, `-c/`	the three exploratory self-play runs (their logs, snapshots and benches kept for diagnosis)
`ppo-2026-08-30-d/`	**the run of record**: `train.log` (one line per update), a snapshot every 250 updates, its benches
`/home/tarstars/prj/troll_farm-local_claude_1/yt_work/ppo/`	the cluster payloads and retrieved outputs (staging, not in git)
`//home/delivery_ml/research/tarstars/troll_farm/runs/`	on the cluster (Cypress): one directory per job — inputs, outputs, logs
`/home/tarstars/nn-venv/`	the Python used for all of this (3.11, PyTorch 2.13 for CPU)
`/home/tarstars/prj/troll_farm/data/raw/games/`	the 24,973 recorded ladder games (7.1 GB), the raw material

4 The structure of the data

A recorded game (one file per game, 24,973 of them) holds the map at turn 0, both players' commands at every turn, and the referee's log. Positions and inventories are not stored per turn; `reconstruct.py` replays both players' commands through our copy of the rules and checks every step against the referee's log — on the 784 top-four games it disagrees with the referee on nothing.

The training set has three kinds of file in one directory:

- *states* (`states-pilot.jsonl.gz`, 12.9 MB): one compact line per turn — every troll's position, talents and cargo, every tree's kind, size, health and fruit, both banks — 58 bytes a turn, 224,400 turns;
- *maps* (`maps-pilot.json`): the 748 boards;
- *labels* (`labels-pilot.npz`, 1.3 MB): what the teacher did. Two kinds of row. A *plan row*, one per turn: which troll the player bought next (the number of that troll among 400 possibilities: speed 1–4, carry 1–5, harvest 0–3, chop 0–4; "nothing" when no purchase follows). A *command row*, one per troll per turn: the move, as one number among 13×242 — 13 kinds of action (move here, harvest, chop, drop, mine, plant four kinds, pick four kinds) over the 242 cells of the largest board; a *move* is labelled with the cell the troll actually reached.

The network's input is not stored: it is built when needed, by the same Rust code the environment uses, as **104 planes of 11×22 numbers** — the cell types, the trees, the trolls and what they carry, distances to the shacks and to the nearest trees, the banks, the scores, the troll being decided, the troll being saved for. (Stored, this would be 20 GB; built on the fly it costs 15 ms a row.)

Why 400 kinds of troll, not delineate's 144. His network chose among speed 1–3, carry 1–4, harvest 0–2, chop 0–3. A count over the top four's 1,725 purchases found 267 outside that box — Bubaptik buys speed-4 trolls in half of its purchases. The game caps nothing, so the vocabulary follows the teachers.

A checkpoint (`*.pt`) is four things: the network's weights, the optimizer's state, the settings it was trained with (including the vocabulary version `v400-2026-08-29` — a checkpoint from another vocabulary is refused), and the step count.

5 The structure of the programs

```
recorded games ---> reconstruct.py ---> build_dataset.py ---> training set
(24,973 files)      (exact turn-by-turn)  (labels + states)    (14 MB)
```

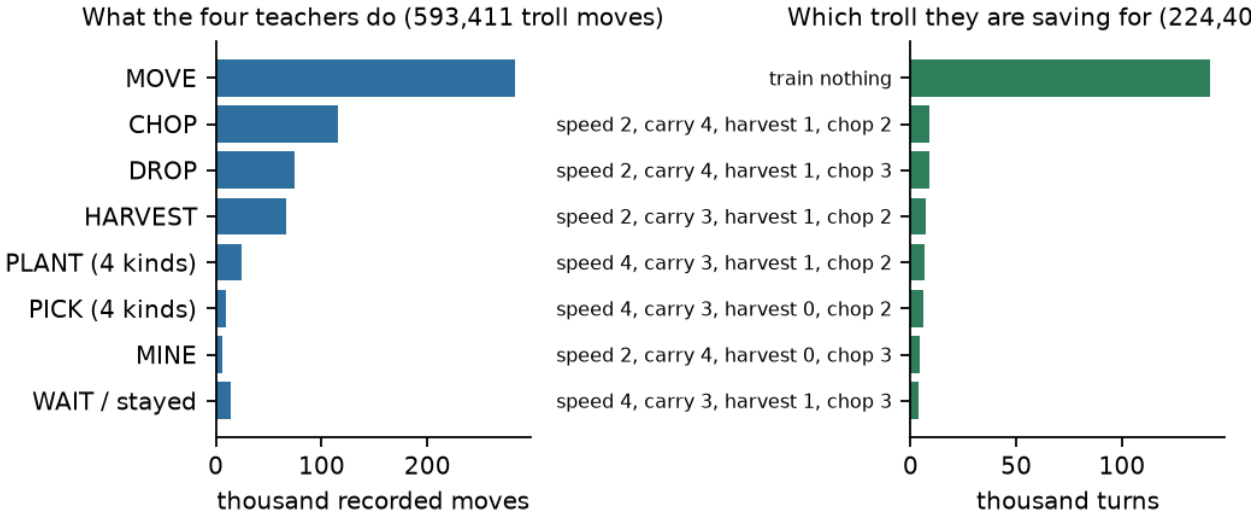
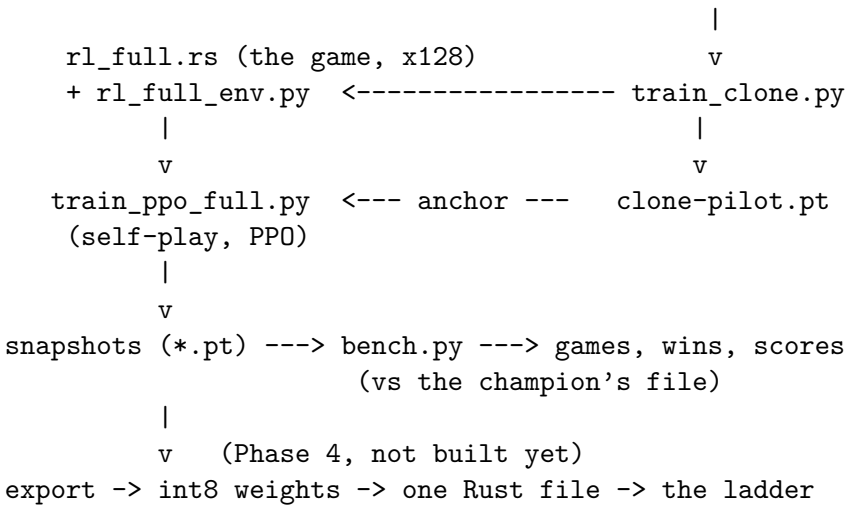


Figure 6: What the training set contains. Left: the moves of the four teachers' trolls. Right: which troll they were saving for, turn by turn ("nothing" after the last purchase of a game).



program	takes	gives
`reconstruct.py`	one recorded game	its exact state at every turn
`build_dataset.py`	a directory of recorded games	the training set (states, maps, labels); a report; a self-check of every label
`train_clone.py`	the training set, the Rust library	the clone checkpoint; a log per pass
`rl_full.rs` + `rl_full_env.py`	maps, opponents, actions	128 games stepping together; rewards; the planes and masks; a replay checker
`train_ppo_full.py`	a starting checkpoint, an anchor checkpoint, the environment	snapshots; one JSON line per update (win rate, margin, speed, how close to the anchor)
`bench.py`	a checkpoint, a compiled bot, a map slice	48 games with scores, purchases, ends; the games saved turn by turn
`nn_runtime.py`	(shared by the three above)	the planes, the masks, the command codec, the exact "can I train" check, the end-of-game rule

The one decision every turn. A turn is several decisions in a row: first *which troll to save for* (one of 400, or nothing); then, for each own troll in order, *what it does* (one of 13×242, most of them masked as impossible); then the turn executes for both players. The reward is the final score difference, paid once per turn on the last decision.

6 The numbers so far

The environment's check	1,000 self-play games replayed through an independent rules copy: 0 disagreements, 0 illegal commands; 214 game-turns/s on the VM's 4 cores
The training set	817,811 decisions (224,400 plan + 593,411 command) from 748 games; 0 labels outside the vocabulary; built in 1 min 42 s
The clone, 4 passes	agreement with the teachers 74 % (which troll) and 65 % (what it does); Figure 3 per move
The clone on the bench	9 wins of 48 vs the champion's file (4 on seat 0, 5 on seat 1); 134 vs 186 points; 0 illegal commands, 0 timeouts; buys a troll at turn 1 in 44 games — the teachers' harvest-carrying trolls, not the champion's chopper
Self-play, run A (exploratory)	1,300 updates, 5.3 million decisions against the old practice mix; wins there 0 → 42 %; **its snapshot vs the champion's file: 2 wins of 48, 87 points to 183** — worse than the clone
Self-play, run C (exploratory)	the fixed trainer, the old mix, 250 updates: **3 wins of 48, 107 to 177** — the same lesson
Self-play, run D (the run of record)	from the clone against the exact champion since 09:42Z; **its update-500 snapshot vs the champion's file: 3 wins of 48, 108 to 184** — the same early dip; a troll bought in 33 games (the clone: 44), no loops
The exact champion opponent	matches the champion's submitted file on 200/200 games and 49,945/49,945 turns, raw and gameplay commands; reproduced; 1,058 turns/s
The cluster	the smoke job: 899 decisions/s on 16 cores (this host: ∼640 on 14 threads); four 12-hour jobs running, 32 cores each
This computer	20 cores, no GPU: the network answers 17,000 boards/s in batches; the bottleneck is building the planes and running the opponents

7 How to read the clone's games

```
cd /home/tarstars/prj/troll_farm
PYTHONPATH=. /home/tarstars/nn-venv/bin/python local_claude_1/nn-bot/bench.py \
    --read local_claude_1/nn-bot/results/clone-2026-08-30-a/\
bench-argmax-replays.jsonl --game 10
```

`-game` is 1 to 48 (0 prints all). Each line is one turn: the champion's commands on the left, the clone's on the right. The table of games (map, seat, scores, purchases, how it ended) is `bench-argmax.json`. Game 10 is a win (225 to 217); game 1 a loss with no purchase (76 to 167); game 34 has the clone's one long loop.

What I saw reading them: the second troll works like a teacher's — walks to trees, harvests, plants lemons, brings fruit home; the first troll often *picks a fruit from the shack and drops it back*, for stretches of ten or twenty turns — copying without a purpose (pick and drop are 13 % of the recorded moves). It does not chop as a plan. Those two habits are what self-play must unlearn first.

8 Risks and what would stop the line

- *Self-play finds a hole in our copy of the game rather than a strategy.* Guard: samples of its games are replayed through the independent rules copy at every check.
- *What the network trains against decides what it learns* — observed today, not feared: three runs against our weak practice bots got worse against the champion. The run of record trains against an exact copy of the champion; the bench uses the champion's real file; the target adds orchard 6; the ladder hour is the truth. The next opponent to add is orchard 6 itself.
- *Speed on the platform.* Our export kernel of July ran a 35,000-weight network in 7 ms a turn; the limit is 50 ms; the network stays small until measured.
- *Dead conditions on the card:* no gain over the clone after 200 million decisions; or a hole in the engine — then the run stops and the card says so.

The agents and the routine

codex_1 and *claude_1* build and reproduce each other's work on the VM; *chatgpt_1* audits on your instruction; I design, rule, integrate onto `main`, train on this computer and write to you. Every ruling and every number is in the card's log; the board's queue item 0 is the milestone note. No platform action has been taken since your word on 08-29 and none will be before Phase 4 and your prediction.